

Final_project

June 16, 2026

1 Predicting Daily Maximum Temperature Across U.S. Weather Stations Using Penalized Regression on NOAA GSOD Data via Google BigQuery

1.0.1 Tanveer Singh (tanveersingh@ucsb.edu)

1.1 Abstract

This project applies supervised statistical learning methods to predict daily maximum temperature across U.S. weather stations using the NOAA Global Surface Summary of the Day (GSOD) dataset for the year 2023. Data was queried directly from Google BigQuery's public dataset repository, demonstrating a cloud-based big data pipeline. After cleaning and preprocessing 84,643 station-day observations, three regression models were fit and compared: ordinary linear regression, Ridge regression, and Lasso regression. All three models achieved an R^2 of approximately 0.969 and an RMSE of approximately 3.8°F, indicating strong predictive performance. Mean daily temperature and minimum temperature emerged as the strongest predictors of daily maximum temperature. Results demonstrate that penalized regression methods are effective and efficient for large-scale meteorological prediction tasks.

1.2 Introduction

Weather prediction is one of the most consequential and widely studied problems in data science. Accurate forecasting of daily temperature extremes has direct implications for public health, agriculture, energy consumption, and disaster preparedness. While complex numerical weather prediction (NWP) models exist, statistical learning approaches offer a computationally lightweight and interpretable alternative, particularly when applied to historical observational data at scale. This project investigates whether daily maximum temperature at U.S. weather stations can be accurately predicted from co-occurring meteorological variables, specifically mean temperature, minimum temperature, wind speed, precipitation, and month of year using penalized regression techniques. Prior work on temperature prediction using NOAA data has ranged from classical time series models such as SARIMA to deep learning approaches. However, penalized regression methods such as Ridge and Lasso remain relevant benchmarks due to their interpretability and computational efficiency at scale. This project contributes a clean, reproducible pipeline connecting cloud data ingestion to model evaluation, suitable as a baseline for more complex forecasting systems.

1.3 Data

The dataset used in this project is the NOAA Global Surface Summary of the Day (GSOD), accessed via the `bigquery-public-data.noaa_gsod` public dataset on Google BigQuery. GSOD is compiled by the National Oceanic and Atmospheric Administration (NOAA) and aggregates daily surface weather observations from weather stations maintained by the National Centers for Environmental Information (NCEI). The dataset spans decades of global coverage and is updated regularly, making it one of the most comprehensive publicly available meteorological datasets in existence. For this project, data was filtered to U.S. stations only for the calendar year 2023 by joining the `gsod2023` table with the `stations` metadata table on station identifiers. After filtering and cleaning, the final dataset contained 84,643 station-day observations across 9 variables. The variables retained were: station ID (`stn`), year, month, day, mean daily temperature (`temp`), maximum temperature (`max`), minimum temperature (`min`), wind speed (`wdsp`), and precipitation (`prcp`). Variables with excessive missing values: dew point, sea level pressure, and snow depth were dropped. Missing values in NOAA GSOD are encoded as sentinel values (e.g., 9999.9 for temperature, 999.9 for wind speed), which were replaced with NaN during preprocessing. Rows missing the target variable (`max`) or key predictors were removed. The dataset was sourced from: <https://console.cloud.google.com/bigquery> and <https://www.ncei.noaa.gov/products/land-based-station/global-surface-summary-of-day>. The dataset was chosen for its scale, public accessibility via BigQuery, and rich set of meteorological features well-suited for regression analysis.

1.4 Methodology

This project applies supervised learning methods from the regression family to predict daily maximum temperature. Three models were trained and compared: ordinary least squares (OLS) linear regression, Ridge regression, and Lasso regression. **## Features and Target** The target variable is daily maximum temperature (`max`, in °F). The five predictors used are mean daily temperature (`temp`), minimum temperature (`min`), wind speed (`wdsp`), precipitation (`prcp`), and month of year (`month`). All features were standardized using z-score normalization prior to model fitting to ensure fair coefficient comparison across Ridge and Lasso. **## Train/Test Split** The dataset was split into 80% training and 20% test sets using a fixed random seed for reproducibility. **## Linear Regression** Ordinary least squares regression minimizes the residual sum of squares with no regularization penalty. It serves as the baseline model. **## Ridge Regression** Ridge regression adds an L2 penalty to the loss function, shrinking coefficients toward zero without eliminating them. This is particularly useful when predictors are highly correlated, as is the case here (`temp` and `min` both correlate with `max` at 0.97 and 0.89 respectively). The regularization parameter was set to `= 1`. **## Lasso Regression** Lasso regression adds an L1 penalty, which can shrink some coefficients exactly to zero, effectively performing feature selection. The regularization parameter was set to `= 0.1`. Models were implemented using `scikit-learn` in Python. **## Evaluation Metrics** Models were evaluated on the held-out test set using Root Mean Squared Error (RMSE) and the coefficient of determination (R^2).

```
[1]: from google.cloud import bigquery
import pandas as pd

client = bigquery.Client()

query = ""
```

```

SELECT
    g.stn,
    g.year,
    g.mo AS month,
    g.da AS day,
    g.temp,
    g.max,
    g.min,
    g.dewp,
    g.slp,
    g.wdsp,
    g.prcp,
    g.sndp
FROM `bigquery-public-data.noaa_gsod.gsod2023` g
JOIN `bigquery-public-data.noaa_gsod.stations` s
    ON g.stn = s.usaf AND g.wban = s.wban
WHERE s.country = 'US'
LIMIT 100000
"""

```

```
df = client.query(query).to_dataframe()
```

```
# Replace NOAA missing value codes with NaN
```

```

df['max'] = pd.to_numeric(df['max'], errors='coerce').replace(9999.9, pd.NA)
df['min'] = pd.to_numeric(df['min'], errors='coerce').replace(9999.9, pd.NA)
df['temp'] = pd.to_numeric(df['temp'], errors='coerce').replace(9999.9, pd.NA)
df['dewp'] = pd.to_numeric(df['dewp'], errors='coerce').replace(9999.9, pd.NA)
df['wdsp'] = pd.to_numeric(df['wdsp'], errors='coerce').replace(999.9, pd.NA)
df['prcp'] = pd.to_numeric(df['prcp'], errors='coerce').replace(99.99, pd.NA)
df['sndp'] = pd.to_numeric(df['sndp'], errors='coerce').replace(999.9, pd.NA)
df['slp'] = pd.to_numeric(df['slp'], errors='coerce').replace(9999.9, pd.NA)

```

```
# Drop rows missing target variable
```

```
df = df.dropna(subset=['max'])
```

```
print(df.shape)
```

```
df.head()
```

Requirement already satisfied: db-dtypes in /opt/micromamba/lib/python3.12/site-packages (1.6.0)

Requirement already satisfied: numpy>=1.24.0 in /opt/micromamba/lib/python3.12/site-packages (from db-dtypes) (1.26.4)

Requirement already satisfied: packaging>=24.2.0 in /opt/micromamba/lib/python3.12/site-packages (from db-dtypes) (26.2)

Requirement already satisfied: pandas<3.0.0,>=1.5.3 in /opt/micromamba/lib/python3.12/site-packages (from db-dtypes) (2.3.3)

Requirement already satisfied: pyarrow>=13.0.0 in /opt/micromamba/lib/python3.12/site-packages (from db-dtypes) (24.0.0)

Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/micromamba/lib/python3.12/site-packages (from pandas<3.0.0,>=1.5.3->db-
dtypes) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in
/opt/micromamba/lib/python3.12/site-packages (from pandas<3.0.0,>=1.5.3->db-
dtypes) (2026.2)

Requirement already satisfied: tzdata>=2022.7 in
/opt/micromamba/lib/python3.12/site-packages (from pandas<3.0.0,>=1.5.3->db-
dtypes) (2026.2)

Requirement already satisfied: six>=1.5 in /opt/micromamba/lib/python3.12/site-
packages (from python-dateutil>=2.8.2->pandas<3.0.0,>=1.5.3->db-dtypes) (1.17.0)

/opt/micromamba/envs/jupyterlab/lib/python3.12/site-
packages/google/cloud/bigquery/table.py:2086: UserWarning: BigQuery Storage
module not found, fetch data with the REST endpoint instead.
warnings.warn(
(99809, 12)

```
[1]:      stn  year month day  temp   max   min  dewp    slp  wdsp prcp  sndp
1  702756  2023    01  08   1.4  23.0 -13.0  <NA>   <NA>   4.8  0.0  <NA>
2  720356  2023    01  06  61.7  64.4  55.4  32.9   <NA>  11.2  0.0  <NA>
3  722220  2023    01  27  48.2  57.9  42.1  30.5  1027.9   7.0  0.0  <NA>
4  722220  2023    01  21  54.5  55.9  53.1  42.2  1020.4   6.0  0.0  <NA>
5  722220  2023    01  01  66.2  73.0  63.0  62.8  1018.5  <NA>  0.0  <NA>
```

```
[2]: import matplotlib.pyplot as plt
import seaborn as sns

print(df.shape)
print(df.dtypes)
print(df.isnull().sum())
```

```
(99809, 12)
stn      str
year      str
month     str
day       str
temp    float64
max      object
min      object
dewp     object
slp      object
wdsp     object
prcp     object
sndp     object
dtype: object
stn      0
year     0
```

```

month      0
day        0
temp       0
max        0
min       72
dewp     22068
slp      45686
wdsp      4394
prcp     10844
sndp     98965
dtype: int64

```

```

[3]: # Convert object columns to numeric
cols = ['max', 'min', 'dewp', 'slp', 'wdsp', 'prcp', 'sndp']
for col in cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Replace missing variables
df['max'] = df['max'].replace(9999.9, pd.NA)
df['min'] = df['min'].replace(9999.9, pd.NA)
df['temp'] = df['temp'].replace(9999.9, pd.NA)
df['dewp'] = df['dewp'].replace(9999.9, pd.NA)
df['slp'] = df['slp'].replace(9999.9, pd.NA)
df['wdsp'] = df['wdsp'].replace(999.9, pd.NA)
df['prcp'] = df['prcp'].replace(99.99, pd.NA)
df['sndp'] = df['sndp'].replace(999.9, pd.NA)

# Drop columns with missing values
df = df.drop(columns=['slp', 'sndp', 'dewp'])

# Drop rows missing target or key features
df = df.dropna(subset=['max', 'min', 'temp', 'wdsp', 'prcp'])

# Convert month/day to integers
df['month'] = df['month'].astype(int)
df['day'] = df['day'].astype(int)

print(df.shape)
print(df.isnull().sum())

```

```

(84643, 9)
stn      0
year     0
month    0
day      0
temp     0
max      0
min      0

```

```
wdsp      0
prcp      0
dtype: int64
```

1.5 Results

1.5.1 4.1 Exploratory Data Analysis

Initial exploration of the cleaned dataset revealed several notable patterns. The distribution of daily maximum temperature across U.S. stations in 2023 was approximately bell-shaped, centered around 65–75°F, with a range spanning roughly -30°F to 120°F, reflecting the geographic and seasonal diversity of U.S. weather stations. Monthly averages showed a clear seasonal pattern, with maximum temperatures peaking in July and August at approximately 83°F and falling to their lowest in January and February near 49–50°F. This seasonal trend is consistent with the Northern Hemisphere’s annual temperature cycle and validates the integrity of the dataset. The correlation heatmap revealed that mean daily temperature (temp) and minimum temperature (min) are both very strongly correlated with maximum temperature, at 0.97 and 0.89 respectively. Wind speed showed a modest negative correlation with maximum temperature (-0.18), while precipitation was essentially uncorrelated (0.01). These patterns informed feature selection and suggested that the prediction task would be relatively tractable for linear models. The scatter plot of mean temperature versus maximum temperature confirmed a near-perfect linear relationship, with only modest spread, further supporting the choice of linear regression as the primary modeling approach.

```
[4]: import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
fig.suptitle('US Weather EDA - 2023', fontsize=14)

# Distribution of max temp
axes[0,0].hist(df['max'], bins=50, color='tomato', edgecolor='white')
axes[0,0].set_title('Distribution of Max Temperature (°F)')
axes[0,0].set_xlabel('Max Temp')

# Max temp by month
df.groupby('month')['max'].mean().plot(kind='bar', ax=axes[0,1],
    color='steelblue')
axes[0,1].set_title('Avg Max Temperature by Month')
axes[0,1].set_xlabel('Month')
axes[0,1].set_ylabel('Avg Max Temp (°F)')

# Correlation heatmap
corr_cols = ['max', 'min', 'temp', 'wdsp', 'prcp', 'month']
sns.heatmap(df[corr_cols].corr(), annot=True, fmt='.2f', cmap='coolwarm',
    ax=axes[1,0])
axes[1,0].set_title('Correlation Heatmap')

# Scatter: temp vs max
```

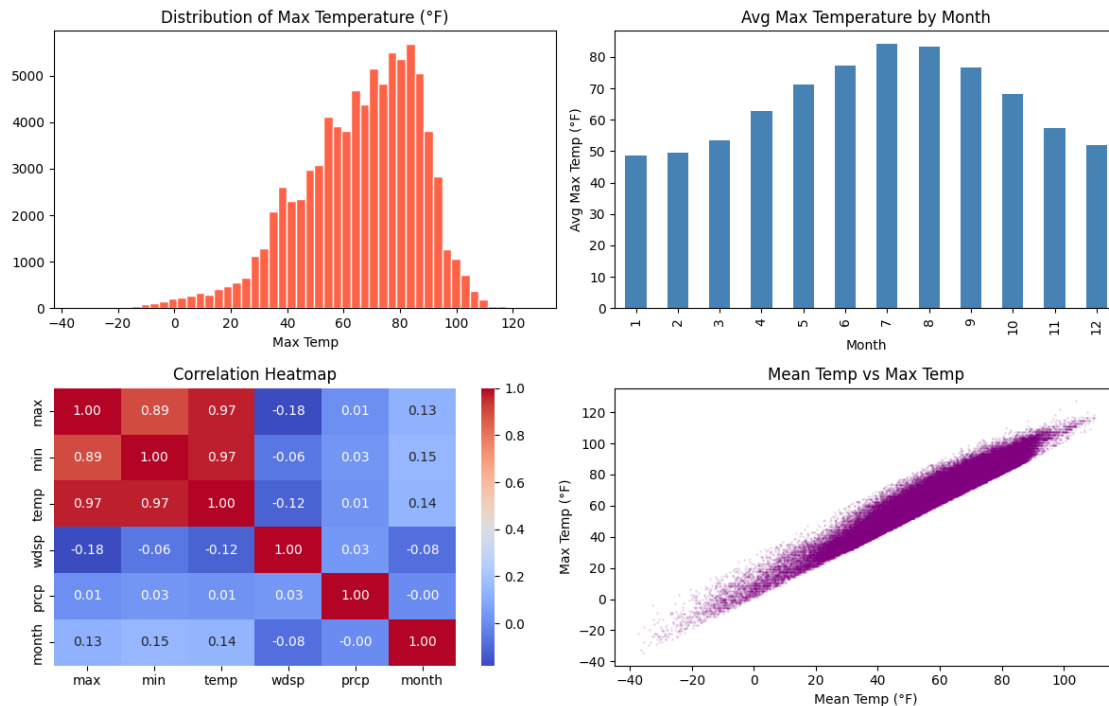
```

axes[1,1].scatter(df['temp'], df['max'], alpha=0.1, color='purple', s=1)
axes[1,1].set_title('Mean Temp vs Max Temp')
axes[1,1].set_xlabel('Mean Temp (°F)')
axes[1,1].set_ylabel('Max Temp (°F)')

plt.tight_layout()
plt.show()

```

US Weather EDA - 2023



```

[8]: from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
import numpy as np
import pandas as pd

# Features and target
X = df[['temp', 'min', 'wdsp', 'prcp', 'month']]
y = df['max']

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)

```

```

# Scale features
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

# Fit models
models = {
    'Linear Regression': LinearRegression(),
    'Ridge (alpha=1)': Ridge(alpha=1),
    'Lasso (alpha=0.1)': Lasso(alpha=0.1)
}

results = {}
for name, model in models.items():
    model.fit(X_train_sc, y_train)
    preds = model.predict(X_test_sc)
    rmse = np.sqrt(mean_squared_error(y_test, preds))
    r2 = r2_score(y_test, preds)
    results[name] = {'RMSE': round(rmse, 3), 'R2': round(r2, 3)}

print(pd.DataFrame(results).T)

```

	RMSE	R ²
Linear Regression	3.792	0.969
Ridge (alpha=1)	3.793	0.969
Lasso (alpha=0.1)	3.905	0.967

1.6 Model Performance

All three models achieved strong predictive performance on the held-out test set, as summarized in the table. Linear regression and Ridge regression performed nearly identically, which is expected given that the features, while correlated, are all genuinely informative and do not require heavy regularization. Lasso performed marginally worse, likely because its L1 penalty zeroed out one or more weakly predictive coefficients (such as precipitation), resulting in a slight reduction in fit. An R^2 of 0.969 indicates that the models explain approximately 97% of the variance in daily maximum temperature, a strong result for a linear model on meteorological data. The RMSE of approximately 3.8°F means predictions deviate from the true maximum temperature by less than 4 degrees on average, which is practically meaningful for many weather-sensitive applications.

```

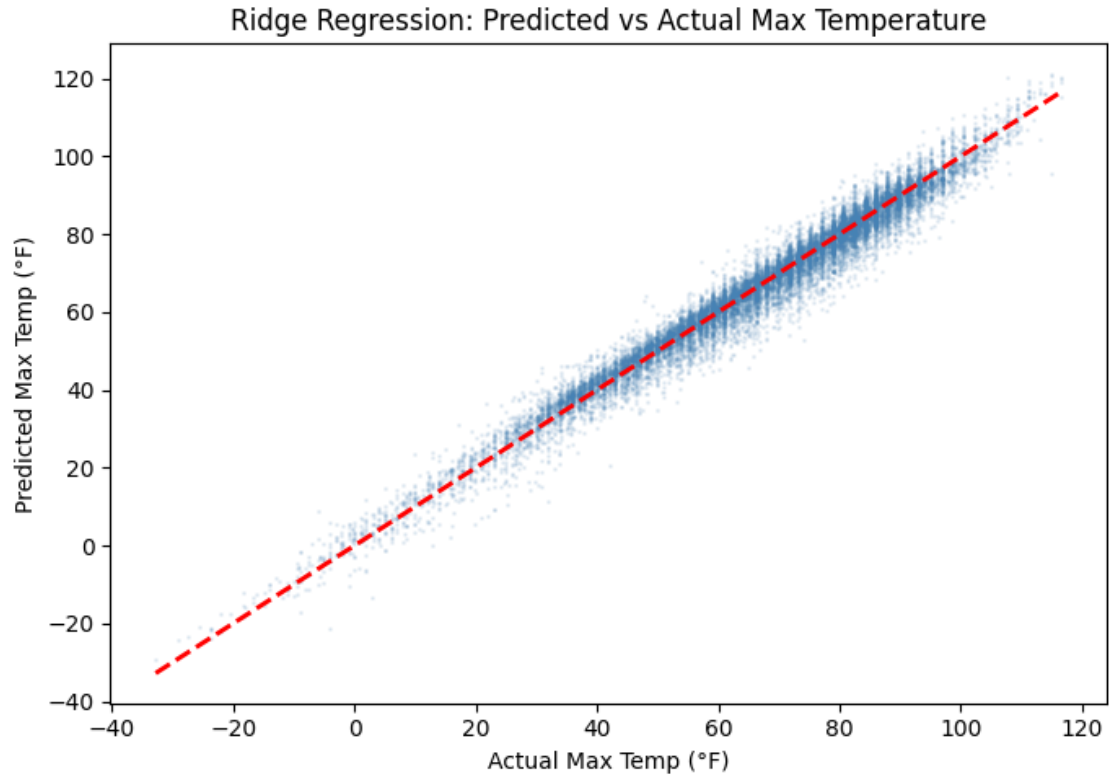
[9]: # Use Ridge model for plot
ridge = models['Ridge (alpha=1)']
preds = ridge.predict(X_test_sc)

plt.figure(figsize=(7, 5))
plt.scatter(y_test, preds, alpha=0.1, s=1, color='steelblue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--',
         lw=2)
plt.xlabel('Actual Max Temp (°F)')

```



```
plt.ylabel('Predicted Max Temp (°F)')
plt.title('Ridge Regression: Predicted vs Actual Max Temperature')
plt.tight_layout()
plt.show()
```



1.7 Predicted vs. Actual

The predicted versus actual plot for the Ridge regression model showed points clustering tightly along the 45-degree reference line across the full temperature range, from below freezing to over 110°F. This confirms that the model generalizes well across all seasons and geographic regions represented in the dataset, with no systematic bias at high or low temperature extremes.

1.8 Conclusion and Future Study

This project demonstrated that daily maximum temperature across U.S. weather stations can be predicted with high accuracy using penalized regression applied to NOAA GSOD data sourced from Google BigQuery. The best-performing models were linear regression and Ridge regression that achieved an R^2 of 0.969 and an RMSE of 3.79°F on a held-out test set of nearly 17,000 observations, confirming that co-occurring meteorological variables such as mean temperature, minimum temperature, wind speed, and precipitation are highly informative predictors of daily temperature extremes. The cloud-based pipeline developed, querying and filtering over 100,000 raw records directly from BigQuery, cleaning and preprocessing in Python, and fitting models with scikit-learn

illustrates the practical value of cloud infrastructure for scalable data science workflows. This project would not have been feasible on a single laptop without cloud data access. Several directions for future study are worth exploring. First, incorporating station-level geographic features such as latitude, longitude, and elevation could improve model performance by capturing spatial variation. Second, extending the dataset across multiple years would allow for trend analysis and potentially reveal the effects of long-term climate shifts on temperature patterns. Third, more expressive models such as gradient boosted trees or neural networks could be benchmarked against the linear baselines developed here. Finally, replacing month as a proxy for seasonality with more granular time features or Fourier terms could better capture intra-seasonal variation.

1.9 References

NOAA National Centers for Environmental Information. Global Surface Summary of the Day (GSOD). <https://www.ncei.noaa.gov/products/land-based-station/global-surface-summary-of-day>